Task 13: Morphological Dilation

Expands foreground feature boundaries across a binary image utilizing a custom square structuring element.

This program performs Morphological Dilation using Python. It is an important Morphological Image Processing technique used in Digital Image Processing.

The main purpose of this program is to increase the size of white foreground objects in a binary image.

Your program performs four main operations:

1. Load the Original Grayscale Image.
2. Convert the Grayscale Image into a Binary Image.
3. Apply Morphological Dilation using a 3×3 Kernel.
4. Display and Save the Dilated Image.

1. What is Image Dilation?

Dilation is a Morphological Operation that increases the size of foreground objects in a binary image.

In a Binary Image:

- White Pixel (255) = Foreground or Object.
- Black Pixel (0) = Background.

Dilation increases the white object's boundary and makes the object thicker or larger.

Before and After Dilation

Original Object

Smaller White Object

Dilated Object

Larger White Object

Remember:

- Erosion = White Object becomes smaller.
- Dilation = White Object becomes larger.

2. What is a Structuring Element or Kernel?

A Structuring Element is a small matrix used to perform morphological operations.

In your program, a 3×3 Kernel is used.

kernel = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]])

The Kernel looks like this:

3×3 Structuring Element

1	1	1
1	1	1
1	1	1

All nine Kernel positions contain 1.

The Kernel moves across the image and checks the neighboring pixels.

For Dilation, if at least one selected neighboring pixel is White (255), the output pixel becomes White (255).

3. Complete Corrected Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image = Image.open("PNG/Task_13_Morphological_Dilation.png").convert("L")
gray_image = np.array(image)

threshold = 128
height, width = gray_image.shape

binary_image = np.zeros((height, width), dtype=np.uint8)

for i in range(height):
```

```
    for j in range(width):
        if gray_image[i, j] >= threshold:
            binary_image[i, j] = 255
        else:
            binary_image[i, j] = 0

kernel = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
])

kernel_height, kernel_width = kernel.shape
pad_height = kernel_height // 2
pad_width = kernel_width // 2
padded_image = np.zeros(
    (height + 2 * pad_height, width + 2 * pad_width),
    dtype=np.uint8
)

padded_image[
    pad_height:pad_height + height,
    pad_width:pad_width + width
] = binary_image

dilated_image = np.zeros((height, width), dtype=np.uint8)
```

```
for i in range(height):
    for j in range(width):
        for ki in range(kernel_height):
            for kj in range(kernel_width):

                if kernel[ki, kj] == 1:

                    if padded_image[i + ki, j + kj] == 255:
                        dilated_image[i, j] = 255

plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)
plt.imshow(gray_image, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(binary_image, cmap='gray')
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(dilated_image, cmap='gray')
plt.title("Dilated Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()


Image.fromarray(dilated_image).save("Output/dilated_output.png")


print("Morphological Dilation Completed Successfully!")
```

4. Code Explanation (Step by Step)

Step 1: Import Libraries

import numpy as npimport matplotlib.pyplot as pltfrom PIL import Imageimport os

Here, we import four libraries.

- numpy: Used for creating arrays and performing mathematical operations.
- matplotlib.pyplot: Used to display images.
- PIL: Used to open and process images.
- os: Used to create the output folder.

Step 2: Load Grayscale Image

image = Image.open("PNG/Task_13_Morphological_Dilation.png").convert("L")gray_image = np.array(image)

First, Image.open() loads the original image from the specified location.

Then, convert("L") converts the image into Grayscale format.

Finally, np.array() converts the image into a NumPy array.

Now, we can access individual pixel values.

Step 3: Set Threshold Value

threshold = 128height, width = gray_image.shape

Here, we set the Threshold Value to 128.

Thresholding is used to convert a Grayscale Image into a Binary Image.

The image dimensions are also calculated.

- height = Number of rows.

- width = Number of columns.

Step 4: Create Binary Image

binary_image = np.zeros((height, width), dtype=np.uint8)

Here, we create a blank binary image with the same dimensions as the original image.

Initially, all pixel values are 0, meaning black.

Then we use nested loops:

for i in range(height): for j in range(width): if gray_image[i, j] >= threshold: binary_image[i, j] = 255 else: binary_image[i, j] = 0

Here, every pixel is checked against the threshold value.

There are two conditions:

- If the pixel value is greater than or equal to 128, it becomes 255 (White).
- Otherwise, it becomes 0 (Black).

This process is called Image Thresholding.

As a result, the Grayscale Image becomes a Binary Image.

Step 5: Create 3×3 Kernel

kernel = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]])

Here, we create a 3×3 Structuring Element.

All nine values are 1.

This Kernel examines the neighboring pixels around each current pixel.

It determines whether the output pixel should become white during Dilation.

Step 6: Get Kernel Dimensions

kernel_height, kernel_width = kernel.shape

Here, we calculate the Kernel's dimensions.

- kernel_height = Number of rows.
- kernel_width = Number of columns.

Since our Kernel is 3×3, both values are 3.

Step 7: Calculate Padding

pad_height = kernel_height // 2pad_width = kernel_width // 2

Here, we calculate the Padding Size.

Since the Kernel Size is 3×3:

$$pad_height = 3//2 = 1$$

$$pad_width = 3//2 = 1$$

This means we need one extra row and column around the image.

Step 8: Create Padded Image

padded_image = np.zeros((height + 2 * pad_height, width + 2 * pad_width), dtype=np.uint8)

Here, we create a new image with additional zero-valued pixels around its boundary.

The purpose of Padding is to ensure that the Kernel can process pixels near the image boundaries.

Since the padding value is 0, the additional boundary pixels are black.

Step 9: Insert Binary Image into Padded Image

padded_image[pad_height:pad_height + height, pad_width:pad_width + width] = binary_image

Here, the original Binary Image is inserted into the center of the padded image.

The surrounding area remains black.

This allows us to process boundary pixels without going outside the array.

Step 10: Create Empty Output Image

dilated_image = np.zeros((height, width), dtype=np.uint8)

Here, we create a blank output image with the same dimensions as the original image.

Initially, every pixel is black (0).

During Dilation, white pixels will be added to this image.

Step 11: Visit Every Pixel

for i in range(height): for j in range(width):

Here, we use Nested Loops.

The first loop accesses every row.

The second loop accesses every column.

Therefore, every pixel in the image is processed one by one.

Step 12: Access Kernel Neighborhood

for ki in range(kernel_height): for kj in range(kernel_width):

Here, another two loops are used to access every position inside the 3×3 Kernel.

- ki = Kernel row position.
- kj = Kernel column position.

Together, these loops examine all nine Kernel positions around the current image pixel.

Step 13: Apply Dilation Condition

if kernel[ki, kj] == 1: if padded_image[i + ki, j + kj] == 255: dilated_image[i, j] = 255

Most Important Code Section

This is the main logic of Morphological Dilation.

First, the program checks whether the current Kernel position contains 1.

Then it checks whether the corresponding image pixel is white (255).

If at least one selected pixel is white, the output pixel becomes white (255).

Otherwise, it remains black (0).

Remember the rule:

- If ANY selected neighboring pixel is 255 → Output = 255.
- If ALL selected neighboring pixels are 0 → Output = 0.

This is the main difference between Dilation and Erosion.

Step 14: Display Original, Binary and Dilated Images

plt.figure(figsize=(12, 5))

Here, we set the display window size.

Then:

plt.subplot(1, 3, 1)

Creates a layout with one row and three columns.

We display three images:

1. Original Grayscale Image.
2. Binary Image.
3. Dilated Image.

For each image:

plt.imshow(image, cmap='gray')plt.title("Image Title")plt.axis("off")

imshow() displays the image.

cmap='gray' applies a grayscale color map.

title() adds the image title.

axis("off") hides the axes.

Finally:

plt.tight_layout()plt.show()

tight_layout() adjusts spacing between the images.

show() displays the complete result.

Step 15: Save Dilated Image

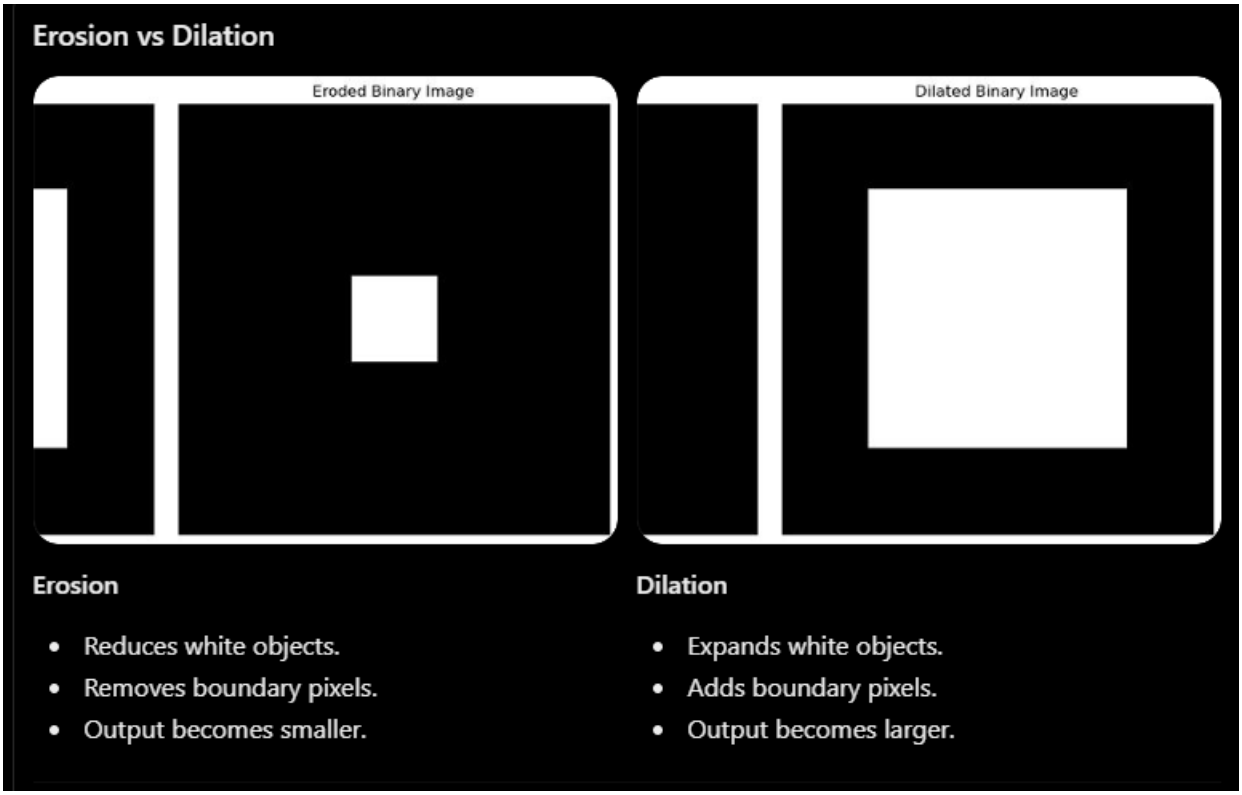
Image.fromarray(dilated_image).save("Output/dilated_output.png")

Here, Image.fromarray() converts the NumPy Array into a PIL Image.

Then save() stores the final Dilated Image in the Output folder.

5. Understand the Main Difference Between Erosion and Dilation

This is very important for your examination.



Main Logical Difference

Erosion:

All selected pixels must be White (255).

Dilation:

At least one selected pixel must be White (255).

বাংলায় ব্যাখ্যা:

প্রথমে আমি প্রয়োজনীয় Library Import করেছি এবং Original Image-টি Grayscale Format-এ Load করেছি। এরপর Thresholding ব্যবহার করে Grayscale Image-কে Binary Image-এ Convert করেছি। এখানে Pixel Value 128 বা তার বেশি হলে White এবং 128-এর কম হলে Black করা হয়েছে।

তারপর একটি 3×3 Structuring Element বা Kernel তৈরি করেছি। এরপর Kernel-এর Dimension বের করে Image-এর চারপাশে Zero Padding দিয়েছি।

পরবর্তীতে একটি Blank Output Image তৈরি করেছি এবং Nested Loop ব্যবহার করে প্রতিটি Pixel Process করেছি। প্রতিটি Pixel-এর জন্য Kernel-এর 9টি Position পরীক্ষা করেছি।

যদি Kernel-এর কোনো Position-এর সঙ্গে মিল থাকা Pixel White হয়, তাহলে Output Pixel White বা 255 করেছি। অন্যথায় Black বা 0 রেখেছি।

এভাবে পুরো Image Process করার ফলে White Foreground Object-এর আকার বৃদ্ধি পেয়েছে। সবশেষে Original Grayscale Image, Binary Image এবং Dilated Image পাশাপাশি Display করেছি এবং Output Image Save করেছি।

Copy Exam Explanation

7. Important Viva Questions and Answers

Q1. What is Image Dilation?

Answer: Dilation is a morphological operation that increases the size of foreground objects in a binary image.

Q2. What is a Structuring Element?

Answer: A Structuring Element is a small matrix used to perform morphological operations on an image.

Q3. Why do we use a 3×3 Kernel?

Answer: To examine nine neighboring pixels around each image pixel.

Q4. What is Thresholding?

Answer: Thresholding converts a grayscale image into a binary image using a predefined threshold value.

Q5. What does 255 represent?

Answer: 255 represents a White Pixel or Foreground Object.

Q6. What does 0 represent?

Answer: 0 represents a Black Pixel or Background.

Q7. What is the purpose of Padding?

Answer: Padding helps process boundary pixels by adding extra pixels around the image.

Q8. What is the difference between Erosion and Dilation?

Answer: Erosion reduces the size of foreground objects, while Dilation increases their size.

Q9. What happens if the Kernel Size increases?

Answer: The foreground objects generally become larger because more neighboring positions are examined.

Q10. What is the purpose of np.zeros()?

Answer: It creates a new array filled with zeros.

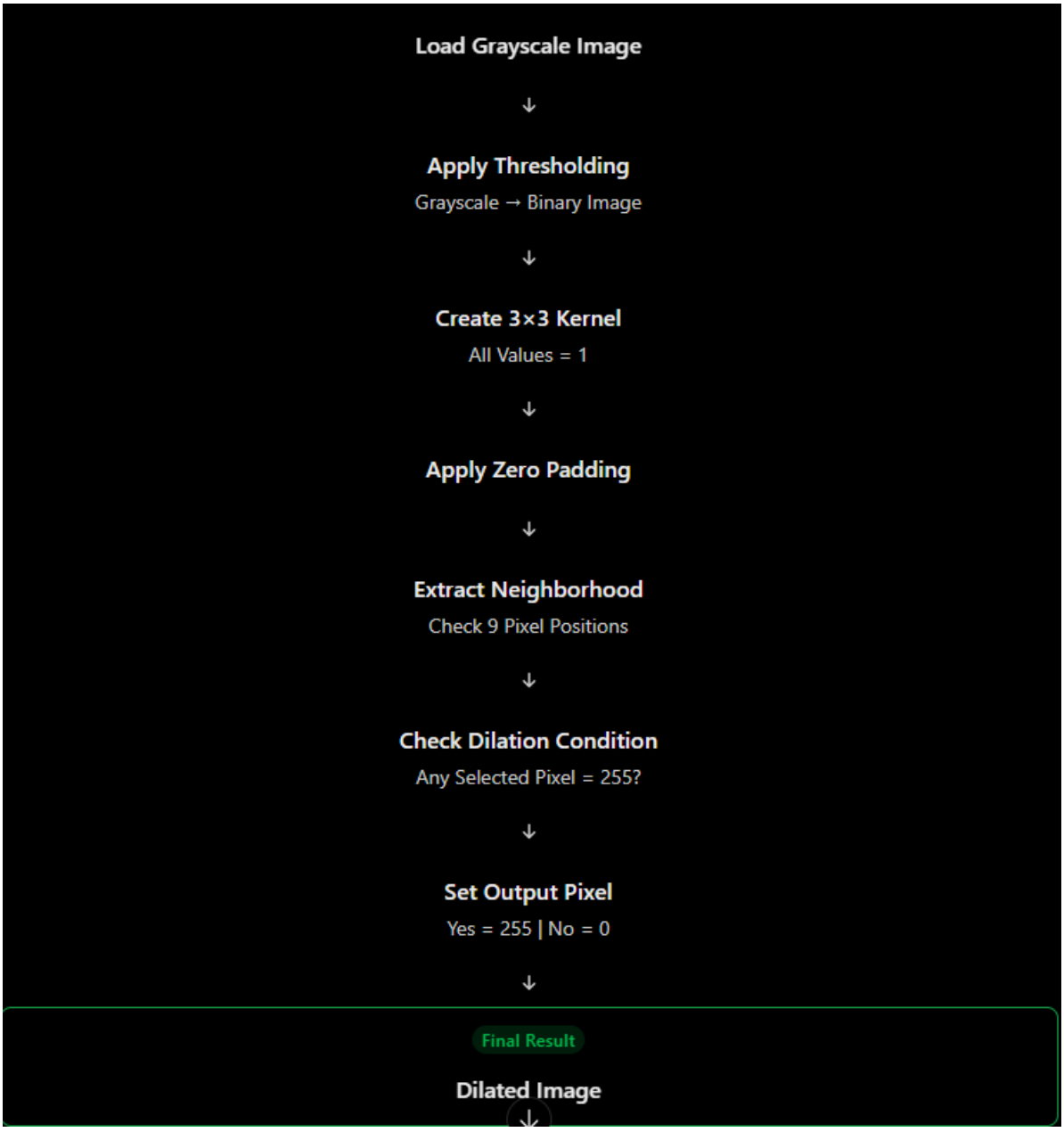
Q11. Why do we use Nested Loops?

Answer: To examine every pixel and its neighboring kernel positions.

Q12. What happens if at least one neighboring pixel is White during Dilation?

Answer: The output pixel becomes White (255).

8. Complete Working Process



Key Caveats:

1. Dilation increases the size of white foreground objects, not necessarily every object in every image.
2. A larger Kernel generally produces stronger dilation.
3. Dilation can connect nearby white objects or close small gaps.
4. The output depends on the threshold value. Here, 128 separates white foreground from black background.
5. Your image is already mostly black and white, so the Binary Image looks very similar to the Original Grayscale Image.

Most Important Code Section for Your Exam:

```
if kernel[ki, kj] == 1:    if padded_image[i + ki, j + kj] == 255:        dilated_image[i, j] = 255
```

Remember this one rule:

Dilation = If ANY selected neighboring pixel is White, the output pixel becomes White.

This is the central concept of your entire program.